

Student 18 **Moniga V** 192521184RC

2 / 2 submitted

Q1. Student Grade Book Application**CODE**

```

students = {}

for i in range(5):
    name = input("Enter student name: ")
    marks = list(map(int, input("Enter 5 subject marks: ").split()))
    students[name] = marks

def grade(avg):
    if avg ≥ 90:
        return "O"
    elif avg ≥ 80:
        return "A+"
    elif avg ≥ 70:
        return "A"
    elif avg ≥ 60:
        return "B+"
    elif avg ≥ 50:
        return "B"
    else:
        return "F"

results = {
    name: {
        "marks": marks,
        "total": sum(marks),
        "average": sum(marks) / len(marks),
        "grade": grade(sum(marks) / len(marks))
    }
    for name, marks in students.items()
}

topper = max(results.items(), key=lambda x: x[1]["average"])

subject_means = []
for i in range(5):
    total = sum(students[name][i] for name in students)
    subject_means.append(total / len(students))

highest_subject = subject_means.index(max(subject_means)) + 1

print("\nStudent Grade Book")
print("-" * 60)
print("{:<10}{:<8}{:<8}{:<8}".format("Name", "Total", "Average", "Grade"))

sorted_results = sorted(results.items(),
                          key=lambda x: x[1]["average"],
                          reverse=True)

for name, data in sorted_results:
    print("{:<10}{:<8}{:<8.2f}{:<8}".format(
        name,
        data["total"],
        data["average"],
        data["grade"]
    ))

print("\nClass Topper:", topper[0], "-", round(topper[1]["average"], 2))
print("Subject with Highest Overall Mean: Subject", highest_subject)

```

INPUT

```

Ravi
98 99 78 77 100
Yogavan
100 100 100 99 98
Moniga
70 69 90 80 88

```

```
kiran
99 78 90 89 99
Divya
99 100 100 100 100
```

OUTPUT

(no output)

Q2. Debugging and Rewriting

CODE

```
accounts = {}

def create_account(acc_no, name, balance):
    accounts[acc_no] = {
        'name': name,
        'balance': balance,
        'txns': []
    }

def deposit(acc_no, amount):
    if acc_no in accounts:
        accounts[acc_no]['balance'] += amount
        accounts[acc_no]['txns'].append(('Deposit', amount))

def withdraw(acc_no, amount):
    if accounts[acc_no]['balance'] >= amount:
        accounts[acc_no]['balance'] -= amount
        accounts[acc_no]['txns'].append(('Withdraw', amount))
    else:
        print("Insufficient balance")

def mini_statement(acc_no):
    for t in accounts[acc_no]['txns'][-5:]:
        print(t)

create_account(1001, "Ravi", 10000)

deposit(1001, 5000)
withdraw(1001, 20000)
withdraw(1001, 3000)

mini_statement(1001)

print("Balance:", accounts[1001]['balance'])
```

OUTPUT

(no output)